

The Graphics Server Protocol

A shared semantic rendering layer for scientific visualization

Nicolas P. Rougier Cyrille Rossant

Draft for discussion - July 2026

Abstract

Scientific visualization libraries expose many different user APIs, but they repeatedly solve the same lower-level rendering problems: batching large numbers of primitives, mapping data to visual attributes, maintaining interactive views, exporting publication figures, supporting remote or headless execution, and adapting to heterogeneous graphics backends. This duplication is especially costly when high-quality GPU rendering is required. The Graphics Server Protocol (GSP) proposes a shared protocol layer for scientific visualization: semantic enough to preserve the meaning of a visual scene, explicit enough to support conformance testing and backend capability discovery, and low-level enough to target fast local GPU rendering, reference CPU rendering, remote renderers, and future web paths. This white paper describes the rationale for GSP, its current design principles, the role of semantic visual families, the importance of explicit capability negotiation and query/readback, and the current reference implementation strategy built around Matplotlib, Datoviz, and VisPy2. It is not an API reference; it is an invitation to discuss the protocol boundary that scientific visualization libraries could share.

Contents

1	Motivation	2
2	Primitive decomposition	3
3	What GSP is not	4
4	The protocol boundary	5
4.1	Control plane and data plane	5
4.2	Transport independence	7
5	Semantic visual families	7
6	Capabilities and adaptation	9

7	Query and readback	10
8	Reference and flagship backends	11
8.1	Matplotlib: reference, conformance, and publication	11
8.2	Datoviz: high-performance GPU rendering	12
8.3	VisPy2: producer API	12
9	Conformance as a design tool	12
10	Large data, extensions, and remote rendering	14
11	Current status	14
12	Relation to existing systems	15
13	Community shape	16
14	Conclusion	16

1 Motivation

Scientific visualization has never had a shortage of plotting APIs. A researcher can choose a high level grammar, a stateful plotting library, a domain-specific viewer, a 3D visualization toolkit, a browser-based charting system, or a low-level GPU library. Matplotlib established a durable model for publication-oriented plotting in Python [2]; VTK demonstrated the value of a large scientific visualization toolkit for geometry, imaging, and 3D pipelines [11]; Vega-Lite showed how far a declarative visualization grammar can go in the browser [10]. These projects differ in user experience, language ecosystem, deployment target, and visual style. That diversity is valuable: no single user API can cover the needs of static publication plots, interactive exploratory tools, microscopy viewers, computational geometry, neural data inspection, volume rendering, and web dashboards.

The problem is lower in the stack. Underneath these different APIs, many libraries rebuild the same renderer-facing concepts. They need to place points, markers, line segments, paths, images, text, meshes, axes, colorbars, cameras, and interactive views. They need to map scalar data through color scales. They need to express coordinate systems and transforms. They need to answer basic scientific questions such as “what item is under this cursor?” or “what data value is represented by this pixel?” They need publication-quality output in one context and fast GPU interaction in another. They need text rendering, antialiasing, color mapping, transparency, and image resampling to be good enough for scientific inspection, not merely sufficient for a demo [1, 4, 6, 7, 9].

They need to run locally, in notebooks, in CI, on remote machines, and eventually in browser or cloud rendering contexts.

Today these concerns are usually packaged as inseparable products: a user API plus a scene model plus a renderer plus an interaction system. Moving a visualization from one renderer to another often means rewriting it. A library author who wants both Matplotlib-quality static output and GPU-speed interaction must either maintain two paths or accept one set of trade-offs. A domain viewer that wants remote rendering must invent its own protocol or adopt a full web stack. A GPU renderer that wants broad adoption must design a full plotting API, even if its real strength is rendering.

GSP is a proposal to separate these layers. Its goal is not to replace plotting libraries. Its goal is to define a protocol surface that different plotting libraries, domain libraries, and renderer implementations can share.

GSP is the layer below plotting APIs and above concrete rendering engines. It describes scientific visual scenes, resources, views, queries, capabilities, and diagnostics in backend-independent terms.

That sentence is the center of the project. A protocol at this level should not decide whether a user writes an object-oriented plotting call, a grammar of graphics, a declarative JSON document, or a domain-specific viewer script. It should decide what semantic scene those producers can hand to a renderer, how a renderer declares what it can faithfully support, and how unsupported behavior is adapted or rejected.

2 Primitive decomposition

Most scientific figures decompose into a limited number of visual components. A scatter plot, for example, is not a magical object from the renderer’s perspective. It is a composition of points or markers, line segments, text glyphs, axes, and filled regions. A high-level plotting API may choose how much of that structure to expose, but the renderer still has to draw the underlying components.

Primitive decomposition is necessary but not sufficient. A protocol cannot simply say “draw points” or “draw text” and be useful to scientific visualization libraries. It also has to define what coordinate space the points live in, what units the marker size uses, how scalar values become colors, what identity a rendered item carries, how the view transforms the data, and what happens when a backend cannot implement the exact request.

GSP therefore treats primitive decomposition as the beginning of the design, not the end. The shared layer must let libraries reuse renderer work without rebuilding a complete renderer engine, while adding three constraints:

- Backend differences must be visible. A renderer must declare capabilities and return diagnostics when it simplifies, adapts, deactivates, or rejects a requested scene.
- Query and readback are not optional debugging features. They are part of the scientific visualization contract.

4 The protocol boundary

The current GSP architecture has four layers:

Layer	Role
Producers	Plotting APIs, VisPy2, domain viewers, notebooks, or application code that create semantic scenes.
GSP session and protocol	Backend-independent scene, resource, view, query, capability, and diagnostic model.
Backend adapters	Translation from GSP semantics to a concrete renderer's objects, commands, state updates, and read-back mechanisms.
Renderers	Matplotlib, Datoviz, remote renderers, browser renderers, or specialized scientific rendering engines.

The protocol is organized around a session between a producer/client and a renderer/server. The server may live in the same process, in a subprocess, on a remote machine, in a browser worker, or behind a cloud service. It exposes capabilities, accepts resources and visual descriptions, executes frames, returns query/readback results, and reports diagnostics. This framing is deliberately inspired by successful protocol layers such as the Language Server Protocol [5]: the important idea is not a particular encoding, but a durable boundary between producers and implementations.

A minimal session has a concrete shape:

1. the producer opens a session and states the protocol versions it can speak;
2. the renderer returns a capability snapshot;
3. the producer creates resources such as buffers, textures, views, and visual descriptions;
4. the producer submits a frame or a retained scene update;
5. the renderer returns a render result with diagnostics and snapshot identifiers;
6. the producer may issue queries against those snapshots and receive structured readback payloads.

The following sketch is intentionally not a Python API commitment. It shows the level of abstraction: the producer describes a scene and asks a renderer to plan it against declared capabilities.

4.1 Control plane and data plane

GSP separates the control plane from the data plane.

The control plane includes session initialization, capability negotiation, scene and visual updates, view and navigation state, transforms, frame submission, query requests, diagnostics, and events. These objects are relatively small and structured. They are the natural domain of schemas, versioned fixtures, debug JSON, and protocol logs.

Listing 1: Illustrative producer-side session sketch.

```
session = gsp.connect("inproc://datoviz")
caps = session.initialize(protocol_versions=["gsp/0.2"])

scene = gsp.Scene(canvas=gsp.Canvas(width=900, height=600))
panel = scene.add_panel(rect=(0, 0, 900, 600), view=gsp.View2D(xlim=(0, 1),
    ylim=(0, 1)))
points = scene.add(
    gsp.PointVisual(
        panel=panel,
        positions=gsp.Array(points_xy, coordinate_space="data"),
        color=gsp.ScalarColor(values, scale="viridis", clim=(0, 1)),
        diameter_px=7,
    )
)

plan = gsp.plan(scene, caps)
result = session.render(plan)
assert result.status in {"accepted", "adapted"}
```

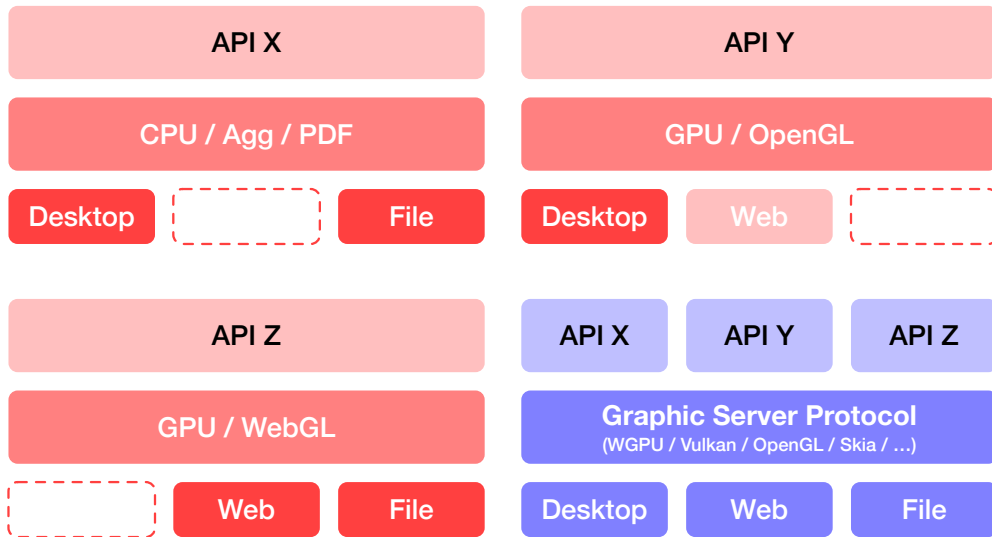


Figure 2: Multiple visualization APIs can target an intermediate rendering protocol rather than binding directly to incompatible rendering engines. GSP adds capability discovery, conformance, query/readback, and transport independence to this shared-protocol structure.

The data plane includes buffers, textures, tiles, chunks, sampled fields, external source handles, cache policy, and level-of-detail descriptors. These objects may be large and should not be forced through text encodings in local workflows. A local desktop renderer should be able to receive NumPy-backed memory, direct buffers, or backend-native resources without paying a serialization tax. A remote renderer may instead receive descriptors that let the server fetch or materialize data under explicit security policy.

This split matters because scientific visualization commonly operates at the boundary where data is too large to copy casually, but interaction still requires low latency.

4.2 Transport independence

The protocol semantics should not depend on one transport. Current design discussions distinguish at least four transport classes:

- in-process transport for fast local rendering;
- binary IPC or shared memory for local process boundaries;
- network transport for remote rendering or server-side data access;
- debug JSON for fixtures, replay, logging, and conformance tests.

This is a practical compromise. JSON fixtures are invaluable for testing and communication. They are not acceptable as the only path for high-performance local rendering. Conversely, an in-process Python adapter is not a sufficient protocol story for remote or polyglot renderers. GSP needs both.

5 Semantic visual families

Scientific visualization primitives can be described as dots, points, markers, segments, paths, glyphs, images, meshes, and volumes. GSP uses the term visual family to emphasize that the protocol contract is semantic, not merely graphical. A visual family defines the meaning of data fields, coordinate spaces, screen-space units, color interpretation, ordering rules, query identity, and backend adaptation behavior.

The current baseline includes the following families and cross-cutting concepts:

Family or concept	Protocol role
Point and marker	Screen-sized circular or shaped items with positions, colors, stroke/fill semantics, and item identity.
Segment and path	One-dimensional stroked geometry with screen-space widths and cap/join semantics.
Image	Raster data placed into a scene, including scalar image interpretation and origin/extent semantics.
Text	Public text as strings with placement, anchoring, rotation, and logical font size; glyph atlases remain backend realization details.
Mesh	Indexed triangle geometry with bounded material/shading semantics and 2D/3D view placement.
Color mapping	Shared scalar-to-color semantics, normalization, named color scales, and colorbar guides.
Guides and layout	Axes, ticks, labels, titles, colorbars, and resolved layout snapshots.
Views and transforms	View2D, View3D, affine transforms, projection, navigation actions, and query inverse payloads.

This list is intentionally narrower than the universe of possible graphics. The point is not to expose every feature of every renderer. The point is to stabilize the families that recur across scientific visualization and make their semantics testable.

A backend is free to realize a point as a shader program, a CPU rasterized collection, a vector artist, a set of instanced quads, or a remote rendering command. What it must not do is silently change the scientific meaning of the point. If requested sizes are in logical screen pixels, the backend must preserve that meaning or report an adaptation. If scalar colors require a particular normalization, the backend must apply it or reject the request. If query support is advertised, the backend must return item identities and payloads that match the visual family contract.

For example, a point-family contract is not just “draw circles.” It contains several semantic commitments:

Point field or behavior	Semantic commitment
Position	Coordinates are interpreted in the visual’s declared coordinate space and through the attached view snapshot.
Diameter	Size is a logical screen-pixel diameter, not data-space radius and not backend marker area.
Color	RGBA values are final colors unless an explicit scalar color encoding is attached.
Identity	Each item has stable visual and item identity for query/readback where advertised.
Adaptation	Unsupported coordinate spaces, color encodings, or query payloads produce diagnostics rather than silent fallback.

A compact debug fixture for such a visual might look like this:

Listing 2: Illustrative debug fixture fragment for a point visual.

```
{
  "visual_family": "point",
  "visual_id": "cells",
  "panel_id": "main",
  "positions": {"resource": "xy", "dtype": "float32", "shape": [1000000, 2],
    "coordinate_space": "data"},
  "diameter_px": 5,
  "color": {"scalar_resource": "cluster_score", "scale": "viridis", "clim":
    [0, 1]},
  "query_payloads": ["visual_id", "item_id", "data_xy", "display_rgba"]
}
```

6 Capabilities and adaptation

Scientific visualization libraries often hide backend differences until a user hits a failure mode: a plot looks different, a field is ignored, an interaction is unavailable, a remote render path silently drops a feature, or a GPU backend produces an image that cannot be queried. GSP takes the opposite position: backend differences are part of the protocol.

Every renderer reports a capability snapshot. Capabilities cover protocol versions, supported visual families, resource limits, texture and buffer formats, transform placement, render target formats, output/export support, query/readback support, layout guarantees, extension support, data source policy, and security posture.

Planning a scene against a backend can then produce one of four outcomes:

Outcome	Meaning
Accept	The backend claims strict support for the requested semantics.
Simplify with diagnostic	The backend can render a simpler but explicit version.
Deactivate with diagnostic	The backend can omit a requested component without failing the entire scene, but only with a visible diagnostic.
Reject with fatal diagnostic	The requested behavior cannot be supported without violating the contract.

This model is essential if GSP is to be trusted. Without explicit adaptation, a protocol becomes a wish list. With explicit adaptation, a renderer can be useful before it is complete, while still being honest about what it did.

Capabilities also let GSP avoid false portability. For example, a renderer may strictly support point rendering but not point picking. It may support static 3D mesh rendering but not triangle picking. It may render guides visually but lack resolved layout snapshots for guide queries. It may support scalar images but not remote tiled data sources. These distinctions matter to library authors and users.

The capability surface therefore needs to be granular. A backend should be able to say “I can render this visual family strictly,” “I can render it only after CPU adaptation,” or “I can render it but cannot answer strict queries.” A planner can then make an explicit decision before any pixels are drawn:

Listing 3: Illustrative capability-driven planning result.

```
request:
  visual: mesh
  coordinates: data3d
  material: flat_lambert
  query: mesh_triangle_pick

datoviz-v0.4 plan:
  render.mesh.data3d: accept
  material.flat_lambert: accept_via_cpu_resolve
  query.mesh_triangle_pick: reject
  diagnostics:
    - code: GSP_QUERY_CAPABILITY_UNSUPPORTED
      detail: native visual_id/primitive_index mapping not advertised
```

This kind of planning result is more useful than a binary backend label. It allows a producer to continue with rendering, disable the picking tool, choose another backend, or show a precise diagnostic to the user.

7 Query and readback

In many graphics APIs, rendering is the main product and picking is an afterthought. Scientific visualization cannot afford that split. Interaction, inspection, annotation, linked views, brushing, tooltips, measurement, and debugging all require the inverse question:

What rendered scene contribution is under this panel coordinate, and what scientific data does it represent?

GSP treats query/readback as first-class protocol behavior. A query result should be able to carry visual identity, item/group/face/voxel/texel identity, coordinate-space information, source data coordinates, displayed color, depth or ordering information, guide hits, and extension payloads where applicable.

This is more than traditional color-buffer picking. The query contract must survive transforms, views, layout, guides, scalar color mapping, remote data sources, and backend adaptation. If a 2D view is panned and zoomed, a query should report through the accepted view snapshot. If a 3D ray is computed from a camera and projection, the result should say which snapshot it used. If a guide tick label is hit, a guide-scoped query should distinguish that from a data visual. If a backend cannot provide these guarantees, it should not advertise them.

The result is a stronger relationship between rendering and interpretation. GSP does

not merely ask a backend to draw. It asks the backend to participate in a scientific visualization session.

A typical query is tied to a rendered snapshot, not just to current mutable state. This prevents a producer from interpreting a hit against a stale camera, layout, or view transform:

Listing 4: Illustrative panel query and response.

```
query:
  panel_id: main
  layout_snapshot_id: layout-42
  view_snapshot_id: view-107
  panel_xy_px: [514, 238]
  scope: all-rendered
  hit_policy: frontmost

response:
  status: hit
  scope: data
  visual_id: cells
  visual_family: point
  item_id: 481516
  data_xy: [0.372, 0.811]
  display_rgba: [68, 1, 84, 255]
  payload:
    scalar_name: cluster_score
    scalar_value: 0.61
```

The exact payload varies by visual family and advertised query capability. The important property is that the result is not merely a pixel color. It is a protocol-level explanation of a rendered contribution.

8 Reference and flagship backends

GSP needs more than a specification. It needs concrete backends that exercise different parts of the design space.

8.1 Matplotlib: reference, conformance, and publication

Matplotlib is the reference backend. That does not mean it is the fastest backend or the only correct backend. It means that it is an excellent place to define conservative semantics, publication output, layout behavior, and CPU-side query or readback paths.

The Matplotlib adapter is valuable precisely because it is not a GPU renderer. It can provide a clear reference implementation for visual families, scalar color mapping, guides, layout snapshots, 2D transforms, navigation actions, and small-scene conformance. It can also produce PNG, PDF, and SVG-style publication artifacts where appropriate.

Backend-native Matplotlib artists remain implementation details; the protocol contract is the GSP scene and result model.

8.2 Datoviz: high-performance GPU rendering

Datoviz is the flagship high-performance GPU backend [8]. It brings the project back to one of its original motivations: high-quality, fast rendering of scientific primitives using modern graphics hardware. The Datoviz v0.4 direction is especially important because its internal scene, panel, visual, camera, query, and capability model is close in spirit to GSP, while still remaining a concrete renderer with its own API and constraints.

The GSP-Datoviz relationship should be understood as an adapter relationship, not as identity. GSP must not expose Datoviz-native handles as protocol semantics. Datoviz may have native controllers, draw states, shader paths, and internal resources, but GSP speaks in terms of visual families, views, resources, capabilities, and query payloads. Where Datoviz can provide strict support, it should advertise it. Where it can only adapt, that adaptation should be explicit. Where the binding or runtime does not expose enough evidence, support should remain unadvertised.

8.3 VisPy2: producer API

VisPy2 is the high-level Python producer target. Its role is different from both Matplotlib and Datoviz. It should give Python users a modern scientific visualization API that can produce GSP scenes. This keeps user ergonomics out of the protocol core. VisPy2 can offer convenience, defaults, domain-friendly helpers, and interactive workflows while relying on GSP for backend planning and rendering.

This separation is a key design discipline. If every user-facing convenience becomes protocol semantics, the protocol becomes too large and too unstable. If the protocol is too low-level, every producer must rebuild the same semantic model. VisPy2 is where that balance can be explored for Python without forcing it on all possible producers.

9 Conformance as a design tool

A protocol without conformance evidence is difficult to discuss. The current project therefore uses fixtures, review packs, capability matrices, diagnostics, and backend audits as part of the design process.

The conformance strategy has several layers:

- protocol fixtures that describe scenes and expected semantics;
- backend capability snapshots that state what is claimed;
- visual review packs that compare rendered outputs across backends;
- structured diagnostics for unsupported or adapted behavior;

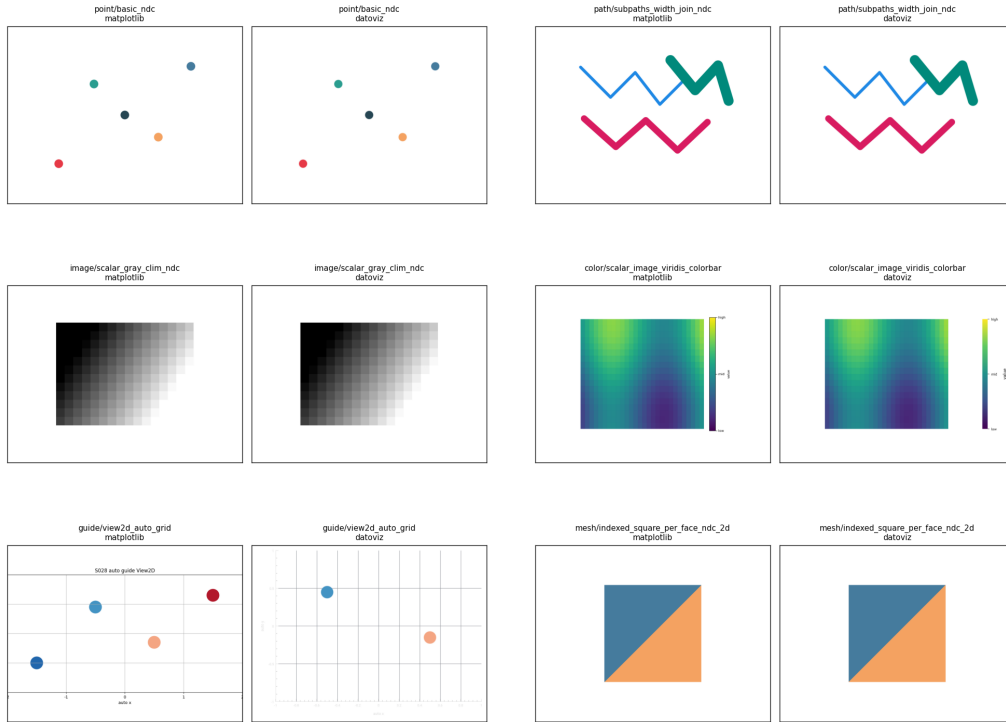


Figure 3: Representative visual review panels from the current implementation work: points, paths, scalar images, color mapping, guides, and meshes. Review packs are not a substitute for formal tests, but they make backend behavior visible across visual families and backends.

- query/readback fixtures where interpretation matters;
- accepted design decisions and ADRs for changes that affect public semantics.

This process is deliberately conservative. A backend should not claim a capability simply because an image looks plausible. Strict support requires matching the semantic contract and, where relevant, matching query/readback behavior. A visually similar guide axis is not the same as resolved layout strictness. A rendered 3D mesh is not the same as proven depth semantics and triangle picking. A GPU path that displays text is not necessarily strict text semantics if anchoring, layout, query, or font metrics are not yet covered.

For a community protocol, this discipline is not bureaucratic. It is what makes collaboration possible. Different projects can implement different subsets while still speaking honestly about their support.

10 Large data, extensions, and remote rendering

Large scientific data should not be treated as ordinary arrays once it exceeds the scale of local eager rendering. Images may be tiled, volumes may be chunked, meshes may have levels of detail, and data may live behind preconfigured sources or remote services. A protocol that assumes every visual field is an eager client-side buffer will fail at exactly the scale where scientific visualization becomes most important.

GSP therefore reserves space for virtual data sources, extension manifests, source locality, server-side fetch descriptors, decoder policy, cache policy, credential policy, and progressive refinement. The current conservative posture is intentionally security-conscious: no dynamic extension execution, no arbitrary network egress, no inline secrets, no unbounded materialization, and explicit diagnostics for rejected sources or policies.

Extensions are necessary because no core protocol can anticipate every domain. A microscopy viewer, geospatial viewer, connectomics tool, finite-element mesh viewer, or astronomical data cube may need custom data sources, transforms, visual families, or query payloads. But extensions should not be unstructured escape hatches. They need identifiers, versions, schemas, capability requirements, fallback policy, implementation declarations, and query contracts.

This is one of the places where GSP is intentionally more protocol-like than library-like. A library can add a Python callback and trust the local process. A protocol must say what crosses the boundary, what is safe to execute, what can be cached, what can be fetched, and how failures are reported.

11 Current status

The current implementation is still a research and protocol-development effort, but it is no longer only an idea. The repository now contains a substantial set of accepted specs, ADRs, protocol models, backend adapters, fixtures, review packs, examples, and mission closeouts.

At a high level, the implemented/proven surface includes:

- protocol-owned models for capabilities, command batches, resources, visual families, transforms, views, queries, diagnostics, and fixtures;
- visual family coverage for points, markers, segments, paths, images, text, and meshes;
- scalar color mapping and colorbar guide semantics;
- deterministic View2D navigation and layout-aware guide behavior;
- static View3D, orthographic and perspective camera work, and View3D navigation review paths;
- Matplotlib reference rendering for the core semantic families;
- Datoviz v0.4 adapter work for strict or adapted GPU rendering across many visual families;
- visual QA harnesses, contact sheets, backend capability matrices, and review examples;
- early extension and virtual data-source architecture with conservative security policy.

Equally important, some things remain explicitly not claimed:

- Datoviz support is capability-gated where binding or runtime evidence is incomplete;
- some Datoviz guide/layout behavior remains adapted rather than strict;
- public volume and surface visual families remain future work;
- broad material, lighting, texture, and shader extension models remain deliberately bounded;
- native Datoviz mesh triangle picking is not advertised until public visual/primitive mapping and freshness are proven;
- governance, release process, and cross-project adoption are still open.

This status should be read as a strength rather than a weakness. A protocol proposal becomes more credible when it distinguishes working semantics from aspirations.

12 Relation to existing systems

GSP overlaps with several existing ideas, but it occupies a specific boundary.

Matplotlib backends separate a plotting API from output renderers, but within the Matplotlib artist and figure model [2]. GSP asks whether a protocol layer can sit below multiple scientific visualization APIs, not only one.

Vega and Vega-Lite demonstrate the power of declarative visualization specifications, especially in browser contexts [10]. GSP is lower level and more renderer-oriented. It is not a grammar for charts; it is a protocol for semantic visual scene execution, including local GPU paths and query/readback.

VTK and similar systems provide extensive scientific visualization capabilities, especially for 3D, meshes, and volumes [11]. GSP is not a replacement for a large visualization toolkit. It is a protocol surface that could, in principle, target or interoperate with such renderers for suitable subsets.

ANARI is a relevant comparison for 3D scientific rendering interoperability [3]. GSP is broader in the sense that it targets 2D and 3D scientific visualization, guides, views, query semantics, and producer/backend separation for plotting-like systems. It is narrower in other ways: it is not a general physically based rendering API.

Browser graphics stacks, WebGPU, WebGL, Canvas, SVG, and WebAssembly provide important execution targets [12]. GSP should be able to target browser renderers, but it should not be defined by browser constraints alone. Scientific desktop use still needs a fast local in-process path.

13 Community shape

The long-term value of GSP depends on whether it becomes a shared boundary rather than a private implementation detail. That suggests several community goals:

- keep the core protocol small enough to implement;
- define extensions for specialized domains instead of bloating the core;
- maintain conformance fixtures and capability vocabularies as public artifacts;
- make unsupported behavior explicit and testable;
- invite backend authors to implement subsets without requiring all features;
- invite plotting and domain-library authors to produce GSP scenes without adopting one prescribed user API;
- separate governance of protocol semantics from any single renderer implementation.

The right governance model is still open. A lightweight consortium, a specification repository, versioned extension registry, or working group around scientific visualization interoperability are all plausible. What matters first is a clear protocol thesis and a credible proof that the boundary can work.

14 Conclusion

Scientific visualization needs both diversity and reuse. It benefits from many user APIs, many domain tools, many renderer implementations, and many deployment targets. But it pays a high cost when every project has to rediscover the same renderer-facing semantics in isolation.

GSP proposes a shared layer: not a plotting API, not a renderer, not a file format, but a semantic graphics server protocol for scientific visualization. Its core ideas are

straightforward: describe visual scenes in backend-independent terms; discover capabilities before rendering; adapt or reject explicitly; treat query/readback as part of the scientific contract; preserve a fast local path; and use conformance evidence to make claims testable.

The current implementation work around Matplotlib, Datoviz, and VisPy2 suggests that this boundary is practical. It is not finished. It is now concrete enough to discuss with the wider scientific visualization community.

References

- [1] Roxana Bujack, Terece L. Turton, Francesca Samsel, Colin Ware, David H. Rogers, and James Ahrens. The good, the bad, and the ugly: A theoretical framework for the assessment of continuous colormaps. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):923–933, 2018. doi: 10.1109/TVCG.2017.2743978.
- [2] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science and Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.
- [3] Khronos Group. ANARI: Analytic rendering interface. <https://www.khronos.org/anari/>, 2024.
- [4] Morgan McGuire and Louis Bavoil. Weighted blended order-independent transparency. *Journal of Computer Graphics Techniques*, 2(2):122–141, 2013. URL <http://jcgt.org/published/0002/02/09/>.
- [5] Microsoft and contributors. Language server protocol specification. <https://microsoft.github.io/language-server-protocol/>, 2024.
- [6] Don P. Mitchell and Arun N. Netravali. Reconstruction filters in computer-graphics. *ACM SIGGRAPH Computer Graphics*, 22(4):221–228, 1988. doi: 10.1145/378456.378514.
- [7] J. Anthony Parker, Robert V. Kenyon, and Donald E. Troxel. Comparison of interpolating methods for image resampling. *IEEE Transactions on Medical Imaging*, 2(1):31–39, 1983. doi: 10.1109/TMI.1983.4307610.
- [8] Cyrille Rossant. Datoviz: High-performance interactive scientific data visualization. <https://datoviz.org/>, 2024.
- [9] Nicolas P. Rougier and Behdad Esfahbod. Digital typography: 25 years of text rendering in computer graphics. In *ACM SIGGRAPH 2018 Courses*. Association for Computing Machinery, 2018. doi: 10.1145/3214834.3214837.
- [10] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-Lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030.
- [11] Will Schroeder, Ken Martin, and Bill Lorensen. *The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics*. Kitware, 4 edition, 2006.
- [12] W3C GPU for the Web Community Group. WebGPU. <https://www.w3.org/TR/webgpu/>, 2024.